

# Especificação Técnica do Projeto

## Paralelismo e eficiência energética em busca vetorial: um estudo comparativo entre busca exata e aproximada

Documento-guia com as decisões técnicas fechadas para o grupo e o passo a passo de implementação, do carregamento do dataset aos testes finais.

### 1. Especificações do formulário

#### Aplicação selecionada

Busca ou contagem em vetores (interseção com Aplicações de Aprendizado de Máquina).

#### Descrição e relevância

Implementação em memória compartilhada com OpenMP de duas estratégias de busca por similaridade vetorial (nearest neighbor search): busca exaustiva (brute-force), exata e computacionalmente cara, e busca aproximada via Locality-Sensitive Hashing (LSH), mais rápida mas sujeita a perda de precisão. Bancos vetoriais reais (Qdrant, FAISS, Pinecone) usam variações desse tipo de estrutura em sistemas de recomendação, busca semântica e IA. O trabalho permite observar, num cenário atual, divisão de trabalho, sincronização, redução de resultados parciais e escalabilidade, associados a métricas de desempenho e energia.

#### Pergunta de pesquisa

Como a paralelização com threads afeta o desempenho e a eficiência energética da busca vetorial por similaridade, comparando uma abordagem exata (brute-force) e uma aproximada (LSH), e a partir de quantas threads a busca exaustiva se torna competitiva em tempo de execução com a busca aproximada?

#### Hipótese

O LSH é mais rápido que o brute-force com poucas threads. O brute-force, por ser mais facilmente paralelizável e ter menor overhead de sincronização, escala melhor com o aumento de threads, podendo alcançar ou superar o LSH a partir de um determinado número de núcleos. Em consumo energético, essa vantagem não deve ser proporcional: mesmo que o brute-force alcance o tempo do LSH, seu consumo pode ser maior, dado o volume total de cálculo superior.

#### Métricas

- Tempo de execução (por configuração de threads: 1, 2, 4, 8...)
- Speedup e eficiência paralela
- Consumo energético (medição direta se disponível, ou indicadores indiretos de uso de CPU)
- Recall: proporção de vizinhos corretos do LSH em relação ao ground truth do brute-force

### 2. Stack técnico definido

|                 |                                   |
|-----------------|-----------------------------------|
| Linguagem       | C++                               |
| Paralelismo     | OpenMP (#pragma omp parallel for) |
| Compilador      | g++ com flag -fopenmp             |
| Algoritmo exato | Brute-force (busca exaustiva)     |



```
struct LSHIndex {
    std::vector<Bucket> buckets;
    int num_buckets;
};
```

### Como o brute-force divide o array entre threads

A divisão de trabalho entre threads é pensada em cima do índice do vetor, e só depois convertida em posição de memória — assim nenhuma thread corta um vetor no meio, mesmo em datasets onde a divisão não é exata. Exemplo com 4 threads e  $N = 1.000.000$ :

```
Thread 0 → vetores 0 até 249.999 (posições 0 até 31.999.999)
Thread 1 → vetores 250.000 até 499.999 (posições 32.000.000 até 63.999.999)
Thread 2 → vetores 500.000 até 749.999 (posições 64.000.000 até 95.999.999)
Thread 3 → vetores 750.000 até 999.999 (posições 96.000.000 até 127.999.999)
```

Cada thread calcula a distância do vetor de consulta contra todos os vetores da sua faixa, e guarda o top-K local (os K mais próximos que ela encontrou). No final, um passo sequencial rápido junta os top-K locais das 4 threads e escolhe o top-K global — essa etapa de merge é a operação de redução mencionada no protocolo experimental.

### Como o LSH divide o trabalho entre threads

- **Indexação** (feita uma vez, antes das buscas): cada thread processa uma faixa de vetores e calcula o hash deles. Como vetores de faixas diferentes podem cair no mesmo bucket, é preciso cuidado ao escrever (lock por bucket, ou cada thread acumula resultados numa lista local e mescla nos buckets só no final).
- **Busca**: cada consulta só é comparada com os poucos vetores do bucket correspondente, não com o dataset inteiro — o trabalho por busca é pequeno, então paralelizar dentro de uma única consulta rende pouco. O ganho de threads aparece ao processar muitas queries em paralelo (uma thread por lote de queries) ou na fase de indexação.

Essa diferença importa para a hipótese do trabalho: o brute-force tem trabalho grande e uniforme, fácil de dividir entre threads; o LSH tem trabalho pequeno por busca, mas custo concentrado na indexação — os dois devem escalar de forma diferente com o aumento de threads.

## 4. Passo a passo de implementação

### Etapa 1 — Preparação de dados

- Baixar SIFT1M (ou subconjunto SIFT100K/SIFT10K se o hardware não aguentar o milhão)
- Implementar parser de .fvecs/.ivecs
- Carregar dataset, queries e ground truth para VectorDataset
- Validar carregamento (checar N, D e alguns valores contra o arquivo original)

### Etapa 2 — Brute-force sequencial

- Implementar função de distância euclidiana entre dois vetores
- Implementar busca sequencial: calcular distância da query contra todo o dataset
- Manter top-K com heap (std::priority\_queue) ou ordenação parcial
- Validar resultado contra o ground truth do SIFT1M (recall deve ser 1.0)

### Etapa 3 — Brute-force paralelo (OpenMP)

- Paralelizar o loop de distâncias com #pragma omp parallel for
- Cada thread mantém top-K local da sua fatia do dataset
- Reduzir os top-K locais em um top-K global (etapa de merge)
- Rodar com 1, 2, 4, 8... threads e registrar tempo de execução

- Brute-force não tem fase de indexação — o tempo medido aqui já é tempo de busca puro, e é essa métrica que será comparada com o tempo de busca do LSH (Etapa 5)

#### **Etapa 4 — LSH sequencial**

- Implementar famílias de hash (ex.: projeções aleatórias) e construção dos buckets
- Indexar o dataset inteiro nos buckets (fase única, feita antes das buscas)
- Implementar busca: comparar query só com vetores do(s) bucket(s) correspondente(s)
- Medir recall do LSH contra o ground truth (esperado: menor que 1.0)

#### **Etapa 5 — LSH paralelo (OpenMP)**

- Paralelizar a construção dos buckets (indexação)
- Paralelizar a busca quando houver múltiplas queries ou múltiplas tabelas de hash
- Rodar com a mesma variação de threads da Etapa 3, mesmo dataset e queries

#### **Etapa 6 — Protocolo experimental e métricas**

- Fixar seeds/parâmetros para garantir mesma entrada em todas as configurações
- Rodar cada configuração (algoritmo x threads) múltiplas vezes e tirar média
- Medir o tempo de indexação do LSH separadamente, como dado complementar — não somar ao tempo de busca na comparação principal
- Coletar tempo, speedup, eficiência paralela e recall em uma planilha/CSV
- Medir energia (medidor direto se disponível; senão, uso de CPU como indicador indireto)

#### **Etapa 7 — Análise e escrita do artigo**

- Gerar gráficos: tempo x threads, speedup x threads, recall x algoritmo
- Identificar o ponto em que brute-force paralelo iguala/supera o LSH
- Discutir resultado de energia frente à hipótese
- Escrever artigo (introdução, metodologia, resultados, discussão, referências ABNT)

### **5. Referências bibliográficas iniciais (ABNT)**

AMDAHL, Gene M. Validity of the single processor approach to achieving large scale computing capabilities. In: AFIPS SPRING JOINT COMPUTER CONFERENCE, 1967, Atlantic City. Proceedings [...]. Reston: AFIPS Press, 1967. p. 483-485.

CHAPMAN, Barbara; JOST, Gabriele; VAN DER PAS, Ruud. Using OpenMP: portable shared memory parallel programming. Cambridge: MIT Press, 2007.

GIONIS, Aristides; INDYK, Piotr; MOTWANI, Rajeev. Similarity search in high dimensions via hashing. In: INTERNATIONAL CONFERENCE ON VERY LARGE DATA BASES (VLDB), 25., 1999, Edinburgh. Proceedings [...]. San Francisco: Morgan Kaufmann, 1999. p. 518-529.

HILL, Mark D.; MARTY, Michael R. Amdahl's law in the multicore era. Computer, v. 41, n. 7, p. 33-38, 2008.

MALKOV, Yu A.; YASHUNIN, Dmitry A. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. IEEE Transactions on Pattern Analysis and Machine Intelligence, v. 42, n. 4, p. 824-836, 2020.